

ΕΘΝΙΚΟ ΜΕΤΣΟΒΕΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
Τομέας Τεχνολογίας Υπολογιστών και Υπολογιστών
Εργαστήριο Μικροϋπολογιστών και Ψηφιακών Συστημάτων

Σχεδιασμός Ενσωματωμένων Συστημάτων
9ο Εξάμηνο ΗΜΜΥ

2η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ

Βελτιστοποίηση Δυναμικών Δομών Δεδομένων (Dynamic Data Type Refinement – DDTR) και Dynamic Memory Management (DMM)

Σκοπός της παρούσας άσκησης είναι η βελτιστοποίηση των δυναμικών δομών δεδομένων και της δυναμικής διαχείρισης μνήμης του αλγορίθμου Non-Maximum Suppression (NMS) σε μοντέλο YOLO, με χρήση της μεθοδολογίας «Βελτιστοποίησης Δυναμικών Δομών Δεδομένων» - Dynamic Data Type Refinement (DDTR) και Dynamic Memory Management (DMM). Μέσω των τεχνικών αυτών, οι στόχοι της άσκησης είναι οι εξής:

- Η βελτιστοποίηση του συνολικού χρόνου εκτέλεσής της εφαρμογής NMS (**execution time**).
- Η βελτιστοποίηση των συνολικών προσβάσεων (reads&writes) της κύριας μνήμης (**memory accesses**).
- Η βελτιστοποίηση της μέγιστης ποσότητας κύριας μνήμης που καταλαμβάνει η εφαρμογή (**memory footprint**).

Για την ανάλυση και μελέτη των δομών δεδομένων (data structures) θα χρησιμοποιηθούν τα εξής εργαλεία:

- Η βιβλιοθήκη **DDTR**
- Το εργαλείο **Massif** της **Valgrind Suite**
- Το εργαλείο **perf** του Linux

1. Προετοιμασία για χρήση της βιβλιοθήκης

Κατέβασμα της βιβλιοθήκης Dynamic Data Type Refinement (DDTR)
Μπορείτε να κατεβάσετε τη βιβλιοθήκη DDTR από τον παρακάτω σύνδεσμο:

https://github.com/ManosXen/Embedded-Systems-Second-Exercise

Από το terminal μπορείτε να το κατεβάσετε με git:

git clone https://github.com/ManosXen/Embedded-Systems-Second-Exercise

Το repository περιέχει τη βιβλιοθήκη DDTR (φάκελος ddtr_student/), την εφαρμογή NMS (φάκελος yolo_nms/) και τα scripts εγκατάστασης του perf στην πλακέτα (φάκελος perf_setup/).

Στο directory **ddtr_student/include/** μπορείτε να βρείτε τις υλοποιημένες δομές δεδομένων που υποστηρίζονται και που θα χρησιμοποιήσετε στην παρούσα εργασία, ενώ στο **ddtr_student/examples/** υπάρχουν υλοποιημένα παραδείγματα χρήσης της βιβλιοθήκης.

Εγκατάσταση του εργαλείου Massif της Valgrind Suite

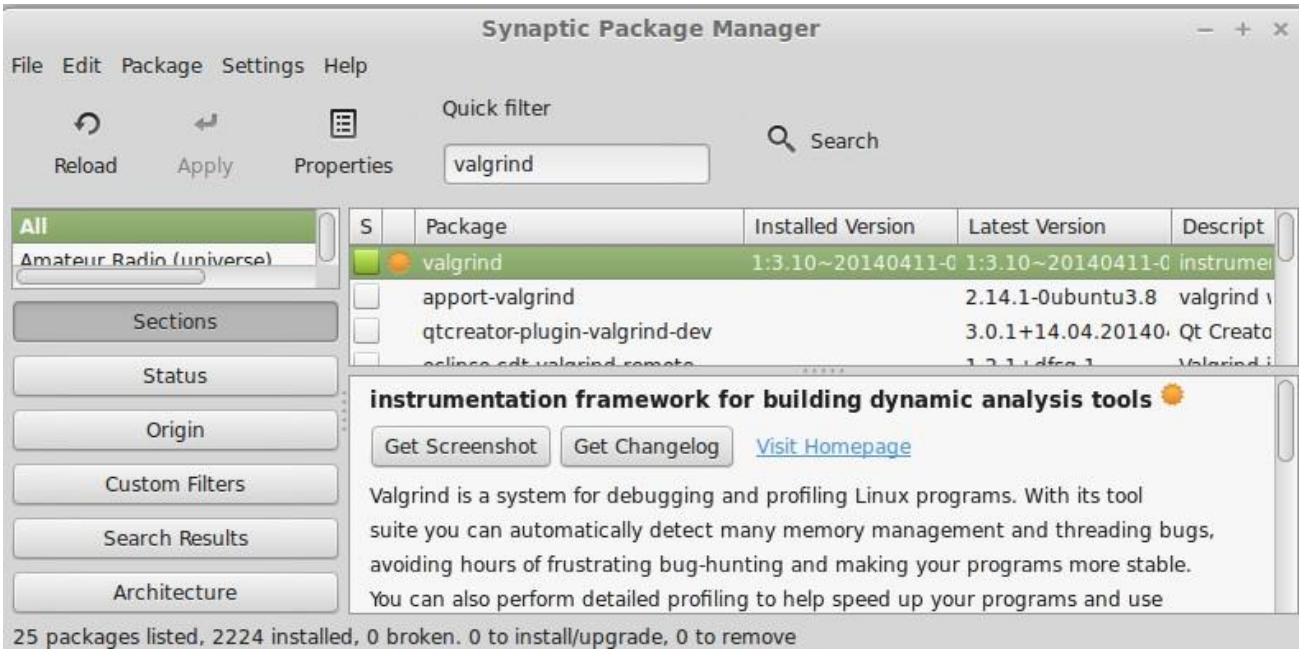
Προαπαιτούμενα: Linux me C++ 17 ή νεότερη

Το Valgrind είναι ένα εργαλείο που παρέχει πληροφορίες για την εφαρμογή, διαβάζοντας το εκτελέσιμο αρχείο και όχι τον source code (i.e. dynamic instrumentation profiling tool). Χρησιμοποιείται για υπολογισμό της χρήσης της μνήμης (memory utilization), των προσβάσεων στη μνήμη (memory accesses), της προβληματικής χρήσης της μνήμης (memory leaks), λαθών σε πολυνηματικές εφαρμογές (threading errors) etc. **Εγκαταστήστε το Valgrind από τον package manager της έκδοσης linux που χρησιμοποιείτε (π.χ. apt-get install valgrind)**. Εναλλακτικά, μπορείτε να το κατεβάσετε και να το εγκαταστήσετε από τον παρακάτω σύνδεσμο:

http://valgrind.org/downloads/current.html

Για την μέτρηση του memory footprint θα χρησιμοποιήσουμε το Massif tool του Valgrind. Για να εκτελέσετε το massif μπορείτε να εκτελέσετε την παρακάτω εντολή για το εκτελέσιμο (executable) το οποίο επιθυμείτε:

valgrind --tool=massif --threshold=0 --peak-inaccuracy=0 --massif-out-file=m.out ./executable



Εγκατάσταση και χρήση του Linux perf

Το **Linux perf** είναι ένα ισχυρό εργαλείο παρακολούθησης και ανάλυσης της απόδοσης (profiling) που βρίσκεται ενσωματωμένο απευθείας στον πυρήνα (kernel) του Linux. Χρησιμοποιείται από προγραμματιστές και διαχειριστές συστημάτων για να εντοπίσουν bottlenecks (σημεία συμφόρησης) στον επεξεργαστή (CPU), στις caches, στην κύρια μνήμη, στις κλήσεις συστήματος (system calls) και στον κώδικα.

Στα πλαίσια της παρούσας εργασίας θα χρησιμοποιηθεί για να μετρήσουμε το πλήθος των προσβάσεων στην κύρια μνήμη του συστήματός μας. Ως προσβάσεις στην κύρια μνήμη λογίζονται το σύνολο των αστοχιών που πραγματοποιούνται στην Last Level Cache (LLC misses).

Εγκαταστήστε το από τον package manager της έκδοσης Linux που χρησιμοποιείτε(π.χ. apt-get install linux-tools-common linux-tools-generic). Για να μετρήσετε τα LLC misses μιας εφαρμογής (executable) αρκεί να εκτελέσετε την παρακάτω εντολή

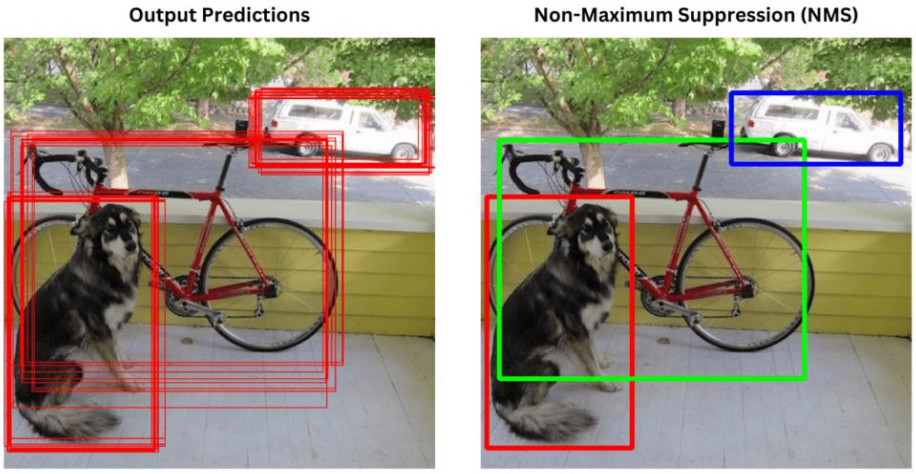
perf stat -e LLC-load-misses,LLC-store-misses ./executable

Το perf μετράει ολόκληρο το πρόγραμμα, δηλαδή και το φόρτωμα των δεδομένων.
Επειδή στο AUP-ZU3 υπάρχουν μόνο δύο επίπεδα cache για να δείτε τα LLC misses θα χρησιμοποιήσετε την εντολή

perf stat -e l2d_cache_refill ./executable

2. Εντοπισμός οχημάτων για φανάρια με το YOLOv5 NMS (github.com/wang-xinyu/tensorrtx).

Ο σύνδεσμος του τίτλου είναι το project tensorrtx, από το οποίο προέρχεται ο αρχικός κώδικας του NMS, και δίνεται μόνο για αναφορά. Δεν χρειάζεται να κατεβάσετε κάτι από εκεί: η εφαρμογή της άσκησης βρίσκεται στον φάκελο yolo_nms/ του repository που κατεβάσατε στην Ενότητα 1.
Τα σύγχρονα συστήματα διαχείρισης κυκλοφορίας μπορούν να αξιοποιούν κάμερες και τεχνικές τεχνητής νοημοσύνης, προκειμένου να προσαρμόζουν δυναμικά τη λειτουργία των φωτεινών σηματοδοτών στις πραγματικές συνθήκες κυκλοφορίας. Σε ένα τέτοιο σύστημα, ένας ενσωματωμένος ελεγκτής (embedded controller) λαμβάνει εικόνες από μία ή περισσότερες κάμερες που είναι εγκατεστημένες σε μια διασταύρωση και υπολογίζει τον αριθμό των οχημάτων που κινούνται σε κάθε κατεύθυνση. Με βάση τα αποτελέσματα αυτά, αποφασίζει πότε πρέπει να αλλάξει η κατάσταση των φωτεινών σηματοδοτών.
Η ταχύτητα επεξεργασίας των δεδομένων είναι ιδιαίτερα σημαντική. Μεγάλες καθυστερήσεις στην επεξεργασία των εικόνων μπορούν να οδηγήσουν σε καθυστερημένη προσαρμογή της σηματοδότησης, μειώνοντας την αποδοτικότητα της κυκλοφορίας. Σε μια ετερογενή πλατφόρμα, όπως η AUP-ZU3, είναι δυνατόν να αξιοποιηθούν διαφορετικές υπολογιστικές μονάδες για την επιτάχυνση της επεξεργασίας. Συγκεκριμένα, το νευρωνικό δίκτυο ανίχνευσης αντικειμένων YOLOv5s μπορεί να εκτελείται στο FPGA, ενώ η CPU αναλαμβάνει την επεξεργασία της εξόδου του δικτύου (post-processing). Το τελευταίο περιλαμβάνει την αποκωδικοποίηση των αποτελεσμάτων (decoding), την εφαρμογή του αλγορίθμου Non-Maximum Suppression (NMS) και την καταμέτρηση των οχημάτων ανά κατεύθυνση.



Το YOLO δεν παράγει απαραίτητα ένα μοναδικό bounding box για κάθε αντικείμενο που εντοπίζει. Αντίθετα, επιστρέφει ένα σύνολο από υποψήφια bounding boxes (candidate boxes). Κάθε υποψήφιο πλαίσιο συνοδεύεται από:

- Την προβλεπόμενη κλάση του αντικειμένου (π.χ. αυτοκίνητο, φορτηγό, άνθρωπος, ποδήλατο).
- Μια τιμή confidence, η οποία εκφράζει τον βαθμό βεβαιότητας της πρόβλεψης.

Ένα πραγματικό αντικείμενο μπορεί να αντιστοιχεί σε πολλά υποψήφια bounding boxes, τα οποία επικαλύπτονται μεταξύ τους. Για την απομάκρυνση αυτών των διπλών ανιχνεύσεων χρησιμοποιείται ο αλγόριθμος Non-Maximum Suppression (NMS). Η βασική λειτουργία του NMS είναι η εξής:

- Τα candidate boxes διαχωρίζονται ανάλογα με την προβλεπόμενη κλάση τους.

- Απορρίπτονται όσα έχουν confidence μικρότερο από ένα προκαθορισμένο κατώφλι (confidence threshold).
- Τα υπόλοιπα ταξινομούνται με φθίνουσα σειρά ως προς το confidence.
- Επιλέγεται το bounding box με το υψηλότερο confidence και απορρίπτονται τα υπόλοιπα boxes που παρουσιάζουν μεγάλη επικάλυψη με αυτό.
- Η διαδικασία επαναλαμβάνεται για τα εναπομείναντα boxes, μέχρι να ολοκληρωθεί η επεξεργασία της συγκεκριμένης κλάσης.

Ο βαθμός επικάλυψης δύο bounding boxes υπολογίζεται με τη μετρική Intersection over Union (IoU). Συνήθως, όταν η τιμή IoU δύο boxes υπερβαίνει ένα προκαθορισμένο κατώφλι (π.χ. 0.45), το box με το χαμηλότερο confidence απορρίπτεται. Ένα σημαντικό χαρακτηριστικό του NMS είναι ότι ο αριθμός των candidate boxes που καλείται να επεξεργαστεί δεν είναι σταθερός, αλλά εξαρτάται από το περιεχόμενο κάθε εικόνας και τις συνθήκες κυκλοφορίας. Επιπλέον, η επιλογή του confidence threshold επηρεάζει άμεσα τον αριθμό των υποψήφιων boxes. Η χρήση χαμηλότερου threshold μπορεί να αυξήσει την πιθανότητα εντοπισμού οχημάτων που είναι μερικώς κρυμμένα ή δεν διακρίνονται καθαρά λόγω δυσμενών συνθηκών, όπως η βροχή ή ο χαμηλός φωτισμός. Ωστόσο, έχει ως αποτέλεσμα την παραγωγή περισσότερων candidate boxes, αυξάνοντας τον υπολογιστικό φόρτο του NMS. Επομένως, η αποδοτική υλοποίηση του NMS είναι σημαντική για τη συνολική απόδοση του συστήματος.

Στο πλαίσιο της άσκησης θα χρησιμοποιήσετε πραγματικά δεδομένα ανίχνευσης οχημάτων, τα οποία έχουν παραχθεί με το μοντέλο YOLOv5s, χρησιμοποιώντας το dataset UA-DETRAC. Συγκεκριμένα, σας παρέχονται τα αποτελέσματα ανίχνευσης για 192 frames, τα οποία προέρχονται από 12 διαφορετικές κάμερες κυκλοφορίας. Για να επικεντρωθείτε αποκλειστικά στη βελτιστοποίηση του αλγορίθμου NMS, τα αποτελέσματα του νευρωνικού δικτύου έχουν υπολογιστεί εκ των προτέρων και αποθηκευτεί στο αρχείο: /data/detrac_y5s_640.bin

Με αυτόν τον τρόπο, δεν απαιτείται η εκτέλεση του YOLOv5s και μπορείτε να αξιολογήσετε τις υλοποιήσεις σας απευθείας πάνω στα δεδομένα εξόδου του δικτύου. Επιπλέον, σε σχέση με την αρχική υλοποίηση που παρέχεται στο GitHub repository, σας δίνεται τροποποιημένο το αρχείο postprocess.cpp, από το οποίο έχει αφαιρεθεί το βήμα της απεικόνισης των τελικών bounding boxes πάνω στις εικόνες. Το συγκεκριμένο βήμα απαιτεί τη χρήση της βιβλιοθήκης OpenCV και δεν είναι απαραίτητο για την αξιολόγηση της απόδοσης του NMS.

Στόχος σας είναι να αντικαταστήσετε το container:

std::vector<Detection>

το οποίο χρησιμοποιείται για την αποθήκευση των candidate boxes κάθε κλάσης μέσα στη συνάρτηση nms() του αρχείου postprocess.cpp. Τα υπόλοιπα στοιχεία της συνάρτησης, συμπεριλαμβανομένων των res και std::map, θα πρέπει να παραμείνουν ως έχουν. Ακολουθώντας τη ροή εκτέλεσης της συνάρτησης nms(), θα παρατηρήσετε ότι: Αρχικά, κάθε candidate box που έχει confidence μεγαλύτερο από το προκαθορισμένο threshold εισάγεται στη λίστα της αντίστοιχης κλάσης. Στη συνέχεια, τα boxes κάθε κλάσης ταξινομούνται με βάση την τιμή confidence. Τέλος, εκτελείται ένας εμφωλευμένος βρόχος (nested loop), ο οποίος συγκρίνει τα bounding boxes και διατηρεί μόνο εκείνα που ικανοποιούν τα κριτήρια του NMS. Η δική σας υλοποίηση θα πρέπει να αντικαταστήσει το παραπάνω container, διατηρώντας τη λειτουργικότητα του αλγορίθμου. Στο πρόγραμμα θα εξετάσουμε confidence threshold = 0.05.

Χρήσιμα tips για την εκπόνηση των παρακάτω ζητουμένων:

- Η συνάρτηση main() του αρχείου main.cpp παρέχεται έτοιμη και αναλαμβάνει την προετοιμασία των δεδομένων και την κλήση της συνάρτησης nms() του αρχείου postprocess.cpp. **Δεν χρειάζεται να τροποποιήσετε τη main()**.
- Η βιβλιοθήκη DDTR παρέχει υλοποίηση για τη sort. Τόσο η sort() όσο και τα δέντρα ταξινομούν με τον operator<, οπότε ορίστε στο postprocess.cpp έναν operator< για το Detection, ώστε τα boxes με το μεγαλύτερο confidence να είναι πρώτα. Για το πώς ορίζεται ένας operator σε μια δική σας δομή, δείτε το tutorial: https://www.learncpp.com/cpp-tutorial/overloading-the-comparison-operators/
- Το κάθε Detection είναι 152 bytes.
- Κατά την εκτέλεση, η nms() καλείται 20 φορές για όλα τα διαθέσιμα frames. Για κάθε frame, καταγράφεται ο χρόνος εκτέλεσης της συνάρτησης σε κάθε επανάληψη και εμφανίζεται ο μικρότερος από τους 20 χρόνους. Η τιμή αυτή εκτυπώνεται με την ένδειξη ms= και εκφράζεται σε milliseconds.

- Για να ελέγξετε ότι η υλοποίησή σας είναι σωστή, κάντε compile με το flag -DVALIDATE και τρέξτε το binary που παράγεται (με κατάληξη -validate). Μετά τον χρόνο τυπώνει validate: OK ή validate: FAILED at frame <n>. Μην μετράτε χρόνο, footprint ή LLC misses με αυτό το binary γιατί προσθέτει overhead.
- Πέρα από το postprocess.cpp και τα αρχεία των allocators δεν χρειάζεται να αλλάξετε κάτι άλλο στον κώδικα.

3. Ζητούμενα

Ο κώδικας γράφεται στο PC σας, ενώ η μεταγλώττιση και όλες οι μετρήσεις εκτός του valgrind στην πλακέτα AUP-ZU3. Συγκεκριμένα:

- **Στο PC:** κατεβάζετε το repository και κάνετε τις αλλαγές στον κώδικα (postprocess.cpp και αρχεία των allocators) και τρέχετε το Massif. Επίσης μπορείτε να κάνετε compile και έλεγχο ορθότητας (-DVALIDATE) και στο PC, για γρηγορότερη debugging.
- **Μεταφορά στην πλακέτα:** αντιγράφετε ολόκληρο τον φάκελο του repository στην κάρτα microSD της πλακέτας, στο /home/xilinx, όπως περιγράφεται στο lab guide.
- **Στην πλακέτα** (μέσω UART): εγκαταστήστε το perf σε περίπτωση που δεν υπάρχει ήδη στην πλακέτα, με τα scripts του φακέλου perf_setup/, και από τον φάκελο yolo_nms/ εκτελείτε το compilation με το build.sh και τις μετρήσεις του perf και του execution time.

Ζητούμενο 1ο: Εκτέλεση της αρχικής έκδοσης της εφαρμογής NMS

Ακολουθήστε τα παρακάτω βήματα για τη μεταγλώττιση και την εκτέλεση του προγράμματος:

- Μεταβείτε στον φάκελο της άσκησης yolo_nms/.
- Μεταγλωττίστε την αρχική υλοποίηση. Για να δημιουργήσετε το executable της αρχικής έκδοσης, εκτελέστε: ./build.sh orig
- Θα δημιουργηθεί το αρχείο bin/nms-orig, το οποίο είναι η αρχική έκδοση της εφαρμογής NMS που θα μελετηθεί σε αυτή την εργασία. Αυτή η εκδοχή θα χρησιμοποιηθεί και ως baseline για τη σύγκριση της απόδοσης της δικής σας υλοποίησης.
- Εκτελέστε το bin/nms-orig και καταγράψτε τον μέσο χρόνο είκοσι εκτελέσεων με το flag -run=20. Ο χρόνος θα εμφανιστεί σε stdout.
- Για να βρείτε τη μνήμη του nms() εκτελέστε την εντολή valgrind --tool=massif --threshold=0 --peak-inaccuracy=0 --massif-out-file=m.out ./bin/nms-orig. Στη συνέχεια, προκειμένου να υπολογιστεί η ακριβής μνήμη του NMS εκτελέστε την εντολή ms_print --threshold=0 m.out και, στο snapshot που σημειώνεται ως (peak), αθροίστε τις γραμμές που αναφέρουν το nms. Το υπόλοιπο heap ανήκει στη main.cpp και στο runtime της C++, και δεν αλλάζει από DDT σε DDT. Καταγράψτε τη συνολική μνήμη που χρησιμοποιεί το nms.
- Οι εκτελέσεις που μετράνε χρόνο και το heap size θα πρέπει να είναι χωριστές γιατί το valgrind προσθέτει overhead.
- Τα executable αρχεία που τρέχουν στο PC σας και στην πλακέτα είναι διαφορετικά και δεν δουλεύουν και στα δύο. Για να έχετε σωστά executable θα πρέπει να κάνετε compile στην πλατφόρμα που θα το εκτελέσετε. Θα δείτε πώς μπορείτε να κάνετε compile στο laptop σας τα executable της πλακέτας (cross-compilation) σε μελλοντικές διαλέξεις.

Ζητούμενο 2ο: Βελτιστοποίηση δυναμικών δομών δεδομένων του αλγορίθμου NMS

Ο source code του NMS είναι στο Github repository που σας δίνεται (https://github.com/ManosXen/Embedded-

Systems-Second-Exercise), στο αρχείο yolo_nms/postprocess.cpp. Θα μελετηθούν οι εξής 9 δομές δεδομένων της βιβλιοθήκης DDTR (σε παρένθεση το όνομα με το οποίο τις επιλέγετε στο build.sh):

- Vector
- Απλά συνδεδεμένη λίστα (SLL) (single_linked_list)
- Διπλά συνδεδεμένη λίστα (DLL) (double_linked_list)
- SLL με roving pointer (SLLR) (roving_single_linked_list)
- DLL με roving pointer (DLLR) (roving_double_linked_list)
- SLL χωρίς sentinel κόμβο (single_linked_list_no_sentinel)
- DLL χωρίς sentinel κόμβο (double_linked_list_no_sentinel)
- SLLR χωρίς sentinel κόμβο (roving_single_linked_list_no_sentinel)
- DLLR χωρίς sentinel κόμβο (roving_double_linked_list_no_sentinel)

Η βιβλιοθήκη περιέχει και δέντρα (π.χ. red_black_tree, b_tree), τα οποία δεν μελετώνται σε αυτή την άσκηση.

Για να εισάγετε τη βιβλιοθήκη στην εφαρμογή, ακολουθήστε τα παρακάτω βήματα (δείτε παράλληλα το postprocess.cpp):

- **Βήμα 1: Εισαγωγή της βιβλιοθήκης.** Η βιβλιοθήκη έχει ήδη γίνει include στο postprocess.cpp (#include <ddtr_library>, γραμμή 8) και στις γραμμές 12–18 ορίζεται ο τύπος Dets = ddtr::sel_dt_dets<Detection, sel_alloc_dets>. Τα sel_dt_dets (το DDT) και sel_alloc_dets (ο allocator) τα ορίζει το build.sh από τα δύο ορίσματά του: π.χ. το ./build.sh double_linked_list glibc δίνει Dets = ddtr::double_linked_list<Detection, ddtr::malloc_allocator>. Έτσι, για να αλλάξετε DDT δεν αλλάζετε τον κώδικα, μόνο το όρισμα του build.sh.
- **Βήμα 2: Αντικατάσταση της δήλωσης.** Στη γραμμή 41 αντικαταστήστε το std::map<float, std::vector<Detection>> m; με std::map<float, Dets> m. Επίσης δείτε τις γραμμές 47 και 48, καθώς ήταν υλοποιημένες πάνω σε vector.
- **Βήμα 3: Ταξινόμηση.** Για την sort της βιβλιοθήκης μας θα πρέπει να οριστεί ο operator< που συγκρίνει δύο Detection. Αυτό γίνεται με το confidence του κάθε candidate box. Έτσι θα πρέπει να κάνετε overload τον operator< για να δουλεύει για Detection. Έπειτα αντικαταστήστε την std::sort(dets.begin(), dets.end(), cmp); της γραμμής 52 με dets.sort(). https://www.learncpp.com/cpp-tutorial/overloading-the-comparison-operators/
- **Βήμα 4: Ο βρόχος με iterators.** Στις λίστες κάθε πρόσβαση με δείκτη (dets[m], dets.begin() + n) διασχίζει τη δομή από την αρχή, γι' αυτό ο βρόχος των γραμμών 53–62 γράφεται με iterators. Η erase(n) επιστρέφει iterator στο επόμενο στοιχείο. Θα πρέπει να προσαρμόσετε αυτόν τον βρόγχο στην βιβλιοθήκη.
- **Βήμα 5: Μεταγλώττιση και έλεγχος.** Μπορείτε να ελέγξετε την ορθότητα της λύσης σας με την εντολή ./build.sh <ddt> glibc -DVALIDATE και ./bin/nms-<ddt>-glibc-validate input=data/detrac_y5s_640.bin, που πρέπει να τυπώσει validate: OK. Για τις μετρήσεις κάντε compile χωρίς το -DVALIDATE: ./build.sh <ddt> glibc, που δημιουργεί το bin/nms-<ddt>-glibc (π.χ. ./build.sh double_linked_list glibc → bin/nms-double_linked_list-glibc).
- Εκτελέστε την εφαρμογή **με όλους τους διαφορετικούς συνδυασμούς υλοποιήσεων δομών δεδομένων για το NMS**. Για κάθε συνδυασμό καταγράψτε τα αποτελέσματά του σχετικά με τον το μέγεθος της

απαιτούμενης μνήμης (**memory footprint**), τον χρόνο εκτέλεσης και τα LLC misses του perf. Για τον χρόνο εκτέλεσης τρέξτε το πρόγραμμα με runs=20 και για για τα LLC τρέξτε με perf stat -r 20 -e LLC-load-misses,LLC-store-misses ./exec . Με runs=0 μπορείτε να δείτε περίπου τα LLC misses που οφείλονται στα άλλα μέρη του προγράμματος.

- Καταγράψτε την έκδοση της εφαρμογής που ελαχιστοποιεί τον χρόνο εκτέλεσης.
- Καταγράψτε την έκδοση της εφαρμογής που ελαχιστοποιεί το συνολικό memory footprint.
- Δημιουργήστε ένα Pareto Plot, στην οποία θα φαίνονται τα σημεία του design space. Ο οριζόντιος άξονας να δείχνει το memory footprint και ο κάθετος άξονας τον χρόνο εκτέλεσης. Πάνω σε αυτό το design space σχεδιάστε και την καμπύλη με τις Pareto optimal λύσεις ως προς το memory footprint και τον χρόνο εκτέλεσης. Προκείμενου να είναι έγκυρες οι μετρήσεις εκτελέστε την κάθε έκδοση της εφαρμογής 3 φορές με 20 runs και καταγράψτε τον μέσο όρο και το variance. Στη γραφική παράσταση πρέπει να συμπεριληφθούν και οι μετρήσεις της αρχικής έκδοσης της εφαρμογής.
- Συγκρίνετε τις βέλτιστες λύσεις κατά Pareto σε σχέση με την αρχική υλοποίηση της εφαρμογής. Σχολιάστε τα αποτελέσματά. Ποια/ες είναι οι βέλτιστες δομές δεδομένων; Εξηγήστε τους λόγους για τους οποίους κάθε μία από αυτές είναι βέλτιστη. Για ποιους λόγους οι συγκεκριμένες επιλογές δομών δεδομένων υπερτερούν των υπολοίπων λύσεων;

Ζητούμενο 3ο: Χρήση διαφορετικών allocators

Μέχρι στιγμής, για τη δυναμική διαχείριση μνήμης γίνεται χρήση της by default έκδοσης της glibc ως allocator. Σε αυτό το ζητούμενο της άσκησης θα εξετάσουμε τη συμπεριφορά της εφαρμογής ως προς τον χρόνο εκτέλεσης και τη χρήση της μνήμης, δοκιμάζοντας διαφορετικούς allocators.

Ο allocator είναι η δεύτερη παράμετρος κάθε DDT της βιβλιοθήκης (README της βιβλιοθήκης, ενότητα 3). Ο allocator είναι ένα struct με δύο static συναρτήσεις, malloc(size) και free(pointer), από τις οποίες το DDT παίρνει και στις οποίες επιστρέφει όλη του τη μνήμη. Στον φάκελο yolo_nms/allocators/ υπάρχει ένα αρχείο για κάθε allocator:

- worst_fit_allocator.hpp: υλοποιημένος, ως παράδειγμα. Παίρνει μνήμη με mmap σε κομμάτια του 1 MB. Μπροστά από κάθε block βάζει ένα header 16 bytes (struct header) με δύο πεδία: το size, το μέγεθος όλου του block μαζί με το header, και το next, έναν δείκτη στο επόμενο ελεύθερο block. Η malloc επιστρέφει τη διεύθυνση αμέσως μετά το header. Η free παίρνει μόνο τον pointer σαν argument και όχι το μέγεθος του block, οπότε το βρίσκει στο header, που βρίσκεται ακριβώς πριν από τον δείκτη ((header*)pointer - 1). Τα ελεύθερα blocks συνδέονται μέσω του next σε μια λίστα (free list) ταξινομημένη κατά διεύθυνση, ενώ τα δεσμευμένα blocks δεν ανήκουν σε καμία λίστα. Η malloc παίρνει το νέο block από το τέλος του μεγαλύτερου ελεύθερου block, και η free, μέσω της insert(), βάζει το block στη σωστή θέση της λίστας και το συγχωνεύει με τα γειτονικά ελεύθερα blocks.
- arena_allocator.hpp, first_fit_allocator.hpp, best_fit_allocator.hpp: οι allocators που θα υλοποιήσετε, με σχόλια TODO στα σημεία που πρέπει να συμπληρώσετε.
- Υλοποιήστε ένα arena allocator, first fit, best fit, συμπληρώνοντας τα TODO στα αντίστοιχα αρχεία, και ξανατρέξτε το προηγούμενο αποτέλεσμα. Για το allocation χρησιμοποιήστε mmap και στην συνέχεια διαχειριστείτε με τον allocator σας τα allocation policies.

Μεταγλωττίστε τη δική σας υλοποίηση. Για να δημιουργήσετε το αντίστοιχο executable, χρησιμοποιήστε το script build.sh με δύο ορίσματα: ./build.sh <ddt> <allocator>, όπου <ddt>: η δομή δεδομένων (ένα από τα 9 ονόματα του Ζητουμένου 2) και <allocator>: ο memory allocator. Οι διαθέσιμοι memory allocators είναι οι εξής: glibc, worst-fit, arena, first-fit και best-fit. Για παράδειγμα, με τον έτοιμο worst fit allocator και τη διπλά συνδεδεμένη λίστα, εκτελώντας στην πλακέτα από τον φάκελο yolo_nms/:

./build.sh double_linked_list worst-fit

θα δημιουργηθεί το εκτελέσιμο:

bin/nms-double_linked_list-worst-fit

Εκτελέστε το πρόγραμμα. Για να εκτελέσετε το παραπάνω binary με τα δεδομένα της άσκησης και confidence threshold ίσο με 0.05, χρησιμοποιήστε την εντολή:

./bin/nms-double_linked_list-worst-fit input=data/detrac_y5s_640.bin conf=0.05

Πριν από τις μετρήσεις ελέγξτε την ορθότητα με ./build.sh double_linked_list worst-fit -DVALIDATE και ./bin/nms-double_linked_list-worst-fit-validate input=data/detrac_y5s_640.bin, που πρέπει να τυπώσει validate: OK. Αντικαταστήστε τα ορίσματα double_linked_list και worst-fit με τη δομή δεδομένων και τον allocator που επιλέξατε.

- Για κάθε συνδυασμό DDT και allocation policy εκτελέστε την εφαρμογή και καταγράψτε τον χρόνο εκτέλεσης, το memory footprint και το πλήθος των προσβάσεων στη μνήμη.

- Σχεδιάστε ένα Pareto plot με όλα τα σημεία του νέου design space για κάθε συνδυασμό DDT και allocator. Στα σημεία να συμπεριληφθούν και οι default υλοποιήσεις που κάνουν χρήση της glibc. Σχεδιάστε το Pareto Front. Τι παρατηρείτε σε σχέση με την αρχική υλοποίηση; Ποιον allocator θα προτιμούσατε και γιατί; Σχολιάστε τα αποτελέσματά σας και αιτιολογήστε τις σχεδιαστικές σας επιλογές.

- Το Massif δεν βλέπει τη μνήμη που γίνεται allocate με την mmap, οπότε κάθε allocator θα πρέπει να μετράει μόνος του τη μνήμη του. Σε αυτό το ερώτημα θα υλοποιήσετε εντός του allocator σας counters που θα μετράνε τα bytes που έγιναν allocate από την mmap και τα byte που χρησιμοποιούν πράγματι οι allocator σας. Αυτοί οι counters θα πρέπει να γίνονται define και να ενημερώνονται εντός των flag -DALLOC_STATS. Κρατήστε τον κώδικα των μετρητών μέσα σε #ifdef ALLOC_STATS και μετρήστε τον χρόνο με binary χωρίς αυτό το flag, ώστε να μην επηρεάσετε της μετρήσεις σας από αυτό το overhead. Τι παρατηρείτε;

Στη συνέχεια κάντε compile με το flag -DALLOC_STATS (π.χ. ./build.sh double_linked_list worst-fit -DALLOC_STATS) και εκτελέστε το ./bin/nms-double_linked_list-worst-fit-stats input=data/detrac_y5s_640.bin. Μετά τον χρόνο τυπώνεται μια γραμμή της μορφής mmap_bytes=1048576 malloc_bytes=178560.1

- Πώς διαχειρίζεται ο allocator σας τα free blocks;

Ζητούμενο 4ο: Χρήση Flag αντί για erase

Η επίδοση και η μνήμη που χρειάζεται το πρόγραμμα εξαρτάται από τα accesses στην μνήμη. Έτσι μία αλλαγή στο πρόγραμμα μπορεί να αλλάξει την συμπεριφορά του άρα και το Pareto Front. Για να το εξετάσετε, θα αλλάξετε τη nms() ώστε να μην κάνει erase τα candidate boxes που απορρίπτονται, αλλά απλά να τα σημαδεύει με ένα flag σε ένα vector. Το vector θα περιέχει ένα unsigned char για κάθε box, με 1 για τα boxes που έχουν απορριφθεί και 0 για τα υπόλοιπα. Τα βήματα είναι:

- **Βήμα 1:** Στο postprocess.cpp, κάτω από τη δήλωση του Dets (γραμμή 18), δηλώστε τον τύπο του vector των flags, ώστε και η μνήμη του να περνά από τον allocator: using Flags = ddtr::vector<unsigned char, sel_alloc_dets>;
- **Βήμα 2:** Κρατήστε τον βρόχο του Ζητουμένου 2 και γράψτε τη νέα εκδοχή μετά το dets.sort(), μέσα σε #ifdef NMS_FLAG (νέα εκδοχή) #else (ο βρόχος με erase) #endif. Το build.sh ορίζει το NMS_FLAG όταν του δώσετε το flag -DNMS_FLAG.
- **Βήμα 3:** Τροποποιήστε τον loop που κάνει erase τα candidate boxes.
- **Βήμα 4:** Ελέγξτε την ορθότητα με ./build.sh <ddt> glibc -DNMS_FLAG -DVALIDATE και ./bin/nms-flag-<ddt>-glibc-validate input=data/detrac_y5s_640.bin, που πρέπει να τυπώσει validate: OK.
- **Βήμα 5:** Επαναλάβετε τις μετρήσεις των Ζητουμένων 2 και 3 με το flag -DNMS_FLAG (τα εκτελέσιμα ονομάζονται bin/nms-flag-<ddt>-<allocator>), για όλους τους συνδυασμούς των 9 DDTs με τους allocators glibc, worst-fit, arena, first-fit και best-fit: χρόνο εκτέλεσης, memory footprint και LLC misses, και για τους δικούς σας allocators το fragmentation (με -DNMS_FLAG -DALLOC_STATS).

Παραδώστε:

- έναν πίνακα με τις μετρήσεις της νέας εκδοχής για κάθε συνδυασμό DDT και allocator. Εξηγήστε για κάθε περίπτωση πως οι σχεδιαστικές επιλογές επηρεάζουν το τελικό αποτέλεσμα.
- ένα Pareto plot (memory footprint – χρόνος εκτέλεσης) με τα σημεία της νέας εκδοχής και, με διαφορετικό σύμβολο, τα σημεία του Ζητουμένου 3, μαζί με την καμπύλη Pareto·
- Ποιο DDT και ποιος allocator είναι τώρα βέλτιστοι; Άλλαξε η επιλογή σε σχέση με τα Ζητούμενα 2 και 3 και γιατί;

Παραδοτέα της άσκησης

Ως παραδοτέο της άσκησης πρέπει να αναρτηθεί εκτενής αναφορά με τα αποτελέσματα της εργασίας και τον σχολιασμό τους, καθώς και ένας φάκελος με όλες τις υλοποιήσεις και τους πηγαίους κώδικες που υλοποιήσατε.

Επεξήγηση όρων

Argument	Όρισμα μιας συνάρτησης
Data Structure Operation Functions	Αναφέρεται στις συναρτήσεις που υλοποιούν τις λειτουργίας μιας δομής δεδομένων. Π.χ. enqueue, dequeue, insert, remove, find, etc.
DDTR (Dynamic Data Type Refinement)	Η μεθοδολογία βελτιστοποίησης δυναμικών δομών δεδομένων
Directory	Κατάλογος του Linux
DLL (Double Linked List)	Διπλά συνδεδεμένη λίστα
DLLR (DLL with Roving Pointer)	Διπλά συνδεδεμένη λίστα με δείκτη στο τελευταίο στοιχείο που προσπελάστηκε.
Sentinel	Κόμβοι στην αρχή και στο τέλος μίας λίστας για να γίνονται γρηγορότερα κάποια operations
Function	Συνάρτηση της γλώσσας C.
Function Definition	Ο ορισμός μιας συνάρτησης (π.χ. int myFun(int x) { return x+1; })
Initialization	Αρχικοποίηση. Εδώ, αναφέρεται στην αρχικοποίηση μιας δομής δεδομένων της βιβλιοθήκης DDTR.
Memory Accesses	Προσβάσεις στη μνήμη
Memory Footprint	Μέγεθος στη μνήμη
SLL (Single Linked List)	Απλά συνδεδεμένη λίστα
SLLR (SLL with Roving Pointer)	Απλά συνδεδεμένη λίστα με δείκτη στο τελευταίο στοιχείο που προσπελάστηκε.